



Guiding Inferences in Connection Tableau by Recurrent Neural Networks

Bartosz Piotrowski^{1,2(✉)} and Josef Urban¹

¹ Czech Institute of Informatics, Robotics and Cybernetics, Prague, Czech Republic

² Faculty of Mathematics, Informatics and Mechanics, University of Warsaw,
Warsaw, Poland

`bartoszpiotrowski@post.pl`

Abstract. We present a dataset and experiments on applying recurrent neural networks (RNNs) for guiding clause selection in the connection tableau proof calculus. The RNN encodes a sequence of literals from the current branch of the partial proof tree to a hidden vector state; using it, the system selects a clause for extending the proof tree. The training data and learning setup are described, and the results are discussed and compared with state of the art using gradient boosted trees. Additionally, we perform a conjecturing experiment in which the RNN does not just select an existing clause, but completely constructs the next tableau goal.

Keywords: Connection tableau · Neural networks · Internal guidance

1 Introduction

There is a class of machine learning sequence-to-sequence architectures based on recurrent neural networks (RNNs) which are successfully used in the domain of natural language processing, in particular for translation between languages [2]. Recently, such architectures proved useful also in various tasks in the domain of symbolic computation [6, 10, 14, 16]. The models *encode* the source sequence to a *hidden vector state* and *decode* from it the target sequence.

In this work, we employ such neural methods to choose among the non-deterministic steps in connection-style theorem proving. In more detail, we want to learn the *hidden proving states* that correspond to the evolving proof trees and condition the next prover steps based on them. I.e., from a set of connection tableau proofs we create a dataset (Sect. 2) of *source-target* training examples of the form (*partial-proof-state*, *decision*) that we then use to train the neural models (Sect. 3). The results are reported in Sect. 4. Section 5 shows an additional experiment with predicting (conjecturing) tableau goals.

B. Piotrowski—Supported by the grant 2018/29/N/ST6/02903 of National Science Center, Poland.

J. Urban—Supported by the *AI4REASON* ERC Consolidator grant nr. 649043 and by the Czech project AI&Reasoning CZ.02.1.01/0.0/0.0/15_003/0000466 and the European Regional Development Fund.

© Springer Nature Switzerland AG 2020

C. Benz Müller and B. Miller (Eds.): CICM 2020, LNAI 12236, pp. 309–314, 2020.

https://doi.org/10.1007/978-3-030-53518-6_23

The connection tableau seems suitable for such methods. The connection proofs grow as branches of a tree rooted in a starting clause. The number of options (clauses) to choose from is relatively small compared to saturation-style provers, where the number of clauses grows quickly to millions during the search. The tableau branches representing the proof states can be the sequential input to the RNNs, which can then decode one or more decisions, i.e., choices of clauses.

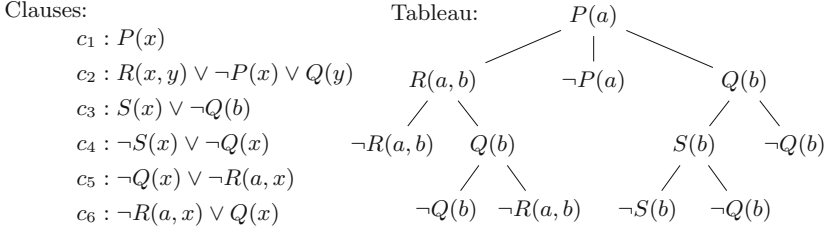


Fig. 1. Closed connection tableau for a set of clauses.

2 A Dataset for Connection-Style Internal Guidance

The experimental data originate from the Mizar Mathematical Library (MML) [7] translated [15] to the TPTP language. We have used the leanCoP connection prover [13] to produce 13822 connection proofs from the Mizar problems.

The connection tableau calculus searches for *refutational proofs*, i.e., proofs showing that a set of first-order clauses is *unsatisfiable*. Figure 1 (adapted from Letz et al. [11]) shows a set of clauses and a *closed connection tableau* constructed from them, proving their unsatisfiability. A closed tableau is a tree with nodes labeled by literals where each branch is *closed*. A *closed* branch contains a pair of *complementary literals* (identical but with opposite polarities). An *open* branch can be extended with descendant nodes by applying one of the input clauses. This *extension* step can often be performed with several different clauses – this is the main non-determinism point. Choosing the correct clause advances the proof, whereas choosing wrongly leads to redundant exploration followed by backtracking.

The training data for choosing good clauses were extracted from the proofs as follows. First, formulas in the proofs were made more uniform by substituting for each universal variable the token **VAR** and for each Skolem function the token **SKLM**. For each non-root and non-leaf node n in each proof tree, we exported two types of paths, which form two kinds of input data for the neural architecture:

- (1) $P_{\text{lits}}(r \rightarrow n)$ – the literals leading from the root r to the node n ,
- (2) $P_{\text{cls}}(r \rightarrow n)$ – the clauses that were chosen on the way from the root r to n .

The output data are created as follows. For each node n we record the decision (i.e., the clause) that led to the proof. Let $\text{clause}(n)$ be the clause selected at node n . For instance, if n is the node labeled by $R(a, b)$ in Fig. 1, $\text{clause}(n) = c_6$.

The pairs $(P_{\text{lits}}(r \rightarrow n), \text{clause}(n))$ and $(P_{\text{cls}}(r \rightarrow n), \text{clause}(n))$ constitute two different sets of training examples for learning clause selection. Each of these sets contains 567273 pairs. Additionally, we have constructed similar data in which the output contains not only the choice of the next clause, but a sequence of two or three such consecutive choices. All these data sets¹ were split into training, validation and testing sets – the split was induced by an initial split of the proofs in proportions 0.6, 0.1 and 0.3, respectively.

3 Neural Modelling and Evaluation Metric

As a suitable sequence-to-sequence recurrent neural model we used an implementation of a neural machine translation (NMT) architecture by Luong et al. [12], which was already successfully used for symbolic tasks in [16] and [14]. All the hyperparameters used for training were inherited from [16].

Let $\text{subsequent_clauses}_i(P_{\text{lits/cls}}(r \rightarrow n))$ be a set of i -long sequences of clauses found in the provided proofs, following a given path of literals/clauses from the root to a node n .² Let $\text{clauses_from_model}_k^i(P_{\text{lits/cls}}(r \rightarrow n))$ be a set of k i -long sequences of clauses decoded from the NMT model (we decoded for $k = 1$ or $k = 10$ most probable sequences using the *beam search* technique [5]). We consider the prediction from the model for a given path of literals/clauses as successful if the sets $\text{subsequent_clauses}_i(P_{\text{lits/cls}}(r \rightarrow n))$ and $\text{clauses_from_model}_k^i(P_{\text{lits/cls}}(r \rightarrow n))$ intersect. The metric of predictive accuracy of the model is the proportion of successful predictions on the test set.

4 Results

The average results for the above metric are shown in Table 1. We can see that predicting the next clause is much more precise than predicting multiple clauses. The accuracy of predicting the next clause(s) from a sequence of clauses is lower than predicting the next clause(s) from a sequence of literals, which means the literals give more precise information for making the correct decision.

We have also investigated how the performance of NMT depends on the length of the input sequences. The results for the

Table 1. Predictive accuracy of the NMT system trained on two types of source paths (literals or clauses), decoding 1–3 consecutive clauses. 1 or 10 best outputs were decoded and assessed.

# clauses to decode	Paths of literals		Paths of clauses	
	1 best output	10 best outputs	1 best output	10 best outputs
1	0.64	0.72	0.17	0.36
2	0.11	0.19	0.03	0.07
3	0.05	0.07	0.01	0.02

¹ The tableau proofs and the sequential training data extracted from it are available at <https://github.com/BartoszPiotrowski/guiding-connection-tableau-by-RNNs>.

² E.g., for the proof from Fig. 1, we have $(c_4) \in \text{subsequent_clauses}_1(P_{\text{lits}}(P(a) \rightarrow S(b)))$, $(c_6, c_5) \in \text{subsequent_clauses}_2(P_{\text{lits}}(P(a) \rightarrow R(a, b)))$, $(c_6) \in \text{subsequent_clauses}_1(P_{\text{cls}}(c_1 \rightarrow c_2))$, or $(c_3, c_4) \in \text{subsequent_clauses}_1(P_{\text{cls}}(c_1 \rightarrow c_2))$.

neural model trained on the paths of literals as the input are shown in the second row of Table 2. As expected, the longer the input sequence, the better is the prediction. The neural model was capable of taking advantage of a more complex context. This differs significantly with the path-characterization methods using manual features (as in [9]) that just average (possibly with some decay factor) over the features of all literals on the path.

To compare with such methods, we trained a classifier based on gradient boosted trees for this task using the XGBoost system [1], which was used for learning feature-based guidance in [9]. To make the task comparable to the neural methods, we trained XGBoost in a multilabel setting, i.e., for each partial proof state (a path of literals) it learns to score all the available clauses, treated as labels. Due to limited resources, we restrict this comparison to the MPTP2078 subset of MML which has 1383 different labels (the clause names).

The average performance of XGBoost on predicting the next clause from the (featurized) path of literals was 0.43. This is lower than the performance of the neural model, also using literals on the path as the input (0.64). The XGBoost performance conditioned on the length of the input path is shown in the third row of Table 2. XGBoost is outperforming NMT on shorter input sequences of literals, but on longer paths, XGBoost gets significantly worse. The performance of the recurrent neural model grows with the length of the input sequence, reaching 0.85 for input length 8. This means that providing more context significantly helps the recurrent neural methods, where the hidden state much more precisely represents (encodes) the whole path. The feature-based representation used by XGBoost cannot reach such precision, which is likely the main reason for its performance flattening early and reaching at most 0.51.

Table 2. Predictive accuracy of the NMT and XGBoost systems for different lengths of input sequences consisting of literals.

Length	1	2	3	4	5	6	7	8
NMT	0.19	0.48	0.64	0.70	0.68	0.72	0.79	0.85
XGB	0.43	0.35	0.42	0.39	0.47	0.41	0.51	0.46

Table 3. Predictive accuracy of conjecturing literals by the NMT system for input sequences of different lengths.

Length	1	2	3	4	5	6	7	all
NMT	0.04	0.05	0.08	0.11	0.14	0.16	0.34	0.08

5 Conjecturing New Literals

As an additional experiment demonstrating the power of the recurrent neural methods we constructed a data set for *conjecturing* new literals on the paths in the tableau proofs. The goal here is not to select a proper literal, but to *construct* it from the available symbols (the number of them for the MML-based data set is 6442). This task is impossible to achieve with the previous methods that can only *rank* or *classify* the available options. Recurrent neural networks are, on the other hand, well-suited for such tasks – e.g., in machine translation, they can learn how to compose grammatically correct and meaningful sentences.

It turns out that this more difficult task is to some extent feasible with NMT. Table 3 shows that NMT could propose the right next literal on the path in a significant number of cases. Again, there is a positive dependence between the length of the input sequence and the predictive performance. Most of the times the correct predictions involve short literals, whereas predicting longer literals is harder. The proposed longer literals often not only do not match the right ones but have an improper structure (see Table 4 for examples of the NMT outputs).

Table 4. Literals conjectured by NMT *vs.* the correct ones. (1) is an example of a correctly predicted output; in (2) NMT was wrong but proposed a literal which is similar to the proper one; (3) shows a syntactically incorrect literal produced by NMT.

	NMT prediction	Correct output
(1)	<code>m1_subset.1(np_1,k4_ordinal1)</code>	<code>m1_subset.1(np_1,k4_ordinal1)</code>
(2)	<code>m1_subset.1(SKLM,k1_zfmisc.1(SKLM))</code>	<code>m1_subset.1(SKLM,SKLM)</code>
(3)	<code>k2_tarski(SKLM,SKLM)=k2_tarski(SKLM)</code>	<code>k2_tarski(SKLM,SKLM)=k2_tarski(SKLM,SKLM)</code>

6 Conclusion and Future Work

In this work, we proposed RNN-based encoding and decoding as a suitable representation and approach for learning clause selection in connection tableau. This differs from previous approaches – both neural and non-neural – by emphasizing the importance of the evolving proof state and its accurate encoding. The approach and the constructed datasets also allow us to meaningfully try completely new tasks, such as automatically conjecturing the next literal on the path. The experimental evaluation is encouraging. In particular, it shows that the longer the context, the more precise the recurrent methods are in choosing the next steps, unlike the previous methods. The evaluation and data sets have focused (as similar research studies [4, 8]) on the machine learning performance, which is known to underlie the theorem proving performance. Future work includes integrating such methods into ATP systems and ATP evaluation similar to [3, 9].

References

1. Chen, T., Guestrin, C.: XGboost: a scalable tree boosting system. *ACM SIGKDD* **2016**, 785–794 (2016)
2. Cho, K., van Merriënboer, B., Gülçehre, Ç., Bahdanau, D., Bougares, F., Schwenk, H., Bengio, Y.: Learning phrase representations using RNN encoder-decoder for statistical machine translation. *EMNLP* **2014**, 1724–1734 (2014)
3. Chvalovský, K., Jakubuv, J., Suda, M., Urban, J.: ENIGMA-NG: efficient neural and gradient-boosted inference guidance for E. *CADE* **27**, 197–215 (2019)
4. Evans, R., Saxton, D., Amos, D., Kohli, P., Grefenstette, E.: Can neural networks understand logical entailment? In: *ICLR 2018* (2018)
5. Freitag, M., Al-Onaizan, Y.: Beam search strategies for neural machine translation. In: *NMT@ACL 2017*, pp. 56–60 (2017)

6. Gauthier, T.: Deep reinforcement learning for synthesizing functions in higher-order logic. CoRR (2019). <http://arxiv.org/abs/1910.11797>
7. Grabowski, A., Kornilowicz, A., Naumowicz, A.: Mizar in a nutshell. J. Formalized Reasoning **3**(2), 153–245 (2010)
8. Kaliszyk, C., Chollet, F., Szegedy, C.: Holstep: a machine learning dataset for higher-order logic theorem proving. In: ICLR 2017 (2017)
9. Kaliszyk, C., Urban, J., Michalewski, H., Olsák, M.: Reinforcement learning of theorem proving. NeurIPS **2018**, 8836–8847 (2018)
10. Lample, G., Charton, F.: Deep learning for symbolic mathematics. CoRR (2019). <http://arxiv.org/abs/1912.01412>
11. Letz, R., Mayr, K., Goller, C.: Controlled integration of the cut rule into connection tableau calculi. J. Autom. Reasoning **13**, 297–337 (1994)
12. Luong, M., Brevdo, E., Zhao, R.: Neural machine translation (seq2seq) tutorial (2017). <https://github.com/tensorflow/nmt>
13. Otten, J., Bibel, W.: leanCoP: lean connection-based theorem proving. J. Symb. Comput. **36**(1–2), 139–161 (2003)
14. Piotrowski, B., Urban, J., Brown, C.E., Kaliszyk, C.: Can neural networks learn symbolic rewriting? CoRR (2019). <https://arxiv.org/pdf/1911.04873.pdf>
15. Urban, J.: MPTP 0.2: Design, implementation, and initial experiments. J. Autom. Reasoning **37**(1–2), 21–43 (2006)
16. Wang, Q., Kaliszyk, C., Urban, J.: First experiments with neural translation of informal to formal mathematics. In: Rabe, F., Farmer, W.M., Passmore, G.O., Youssef, A. (eds.) CICM 2018. LNCS (LNAI), vol. 11006, pp. 255–270. Springer, Cham (2018). https://doi.org/10.1007/978-3-319-96812-4_22